

Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#01)

Apply modern simulation tools ModelSim and Logisim to implement basic processor components

Student Name: Hanzala Saeed Roll No: 24i-6535 Section: CE-A

Date performed: 18-08-2026

MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____

Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Prepared by: Fasih Ahmad

Version: 1.0

Verified by: Dr. Nasir Ali Shah

Updated: Fall 2026

Task 1:

Description:

In this task I wrote Verilog code in ModelSim for basic logic gates using gate-level modeling. These gates includes AND, OR, NOT, NAND, NOR, XOR, XNOR gates. Inputs to the gates are provided by the testbench and outputs are checked using simulation waveforms.

Code:

```
//And gate

module AndGate(a,b,c);

input a;

input b;

output c;

and al(c,a,b);

endmodule

module Andtestbench();

reg x;

reg y;

wire z;

AndGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;
```

```
end

endmodule

//Or gate

module OrGate(a,b,c);

input a;

input b;

output c;

or al(c,a,b);

endmodule

module Ortestbench();

reg x;

reg y;

wire z;

OrGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Not gate

module NotGate(a,c);
```

```
input a;

output c;

not al(c,a);

endmodule

module Nottestbench();

reg x;

wire z;

NotGate uut(x,z);

initial

begin

x=0;

#50 x=1;

#50;

end

endmodule

//Nand gate

module NandGate(a,b,c);

input a;

input b;

output c;

nand al(c,a,b);

endmodule

module Nandtestbench();

reg x;

reg y;
```

```
wire z;

NandGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Nor gate

module NorGate(a,b,c);

input a;

input b;

output c;

nor al(c,a,b);

endmodule

module Nortestbench();

reg x;

reg y;

wire z;

NorGate uut(x,y,z);

initial

begin
```

```
x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Xor gate

module XorGate(a,b,c);

input a;

input b;

output c;

xor al(c,a,b);

endmodule

module Xortestbench();

reg x;

reg y;

wire z;

XorGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;
```

```
#50;

end

endmodule

//Xnor gate

module XnorGate(a,b,c);

input a;

input b;

output c;

xnor al(c,a,b);

endmodule

module Xnortestbench();

reg x;

reg y;

wire z;

XnorGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule
```

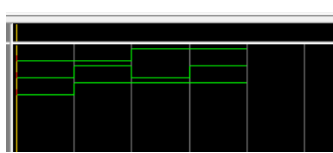
Output:

// Import output here 

And Gate wave form



Or Gate Wave Form



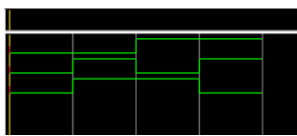
Not Gate wave form



NAND Gate



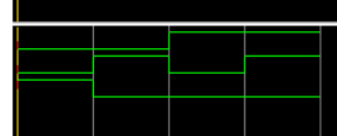
Xor Gate



Xnor Gate



Nor Gate



Task 2:

Description:

In this task I write Verilog code for basic logic gates using dataflow modeling. This involves expressing logic of each gate using boolean expressions and assigning them to an assign statement. Four inputs were given by the testbench to check outputs through wave form.

Code:

```
//And gate

module AndGate(a,b,c);

input a;

input b;

output c;

assign c=a&b;

endmodule

module Andtestbench();

reg x;
```

```
reg y;

wire z;

AndGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Or gate

module OrGate(a,b,c);

input a;

input b;

output c;

assign c=a|b;

endmodule

module Ortestbench();

reg x;

reg y;

wire z;

OrGate uut(x,y,z);

initial

begin

x=0; y=0;
```

```
#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Not gate

module NotGate(a,c);

input a;

output c;

assign c=~a;

endmodule

module Nottestbench();

reg x;

wire z;

NotGate uut(x,z);

initial

begin

x=0;

#50 x=1;

#50;

end

endmodule

//Nand gate

module NandGate(a,b,c);

input a;

input b;
```

```
output c;

assign c=~(a&b);

endmodule

module Nandtestbench();

reg x;

reg y;

wire z;

NandGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Nor gate

module NorGate(a,b,c);

input a;

input b;

output c;

assign c=~(a | b);

endmodule

module Nortestbench();

reg x;

reg y;
```

```
wire z;

NorGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Xor gate

module XorGate(a,b,c);

input a;

input b;

output c;

assign c=a^b;

endmodule

module Xortestbench();

reg x;

reg y;

wire z;

XorGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;
```

```
#50 x=1; y=0;

#50 x=1; y=1;

#50;

end

endmodule

//Xnor gate

module XnorGate(a,b,c);

input a;

input b;

output c;

assign c=~(a^b);

endmodule

module Xnortestbench();

reg x;

reg y;

wire z;

XnorGate uut(x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

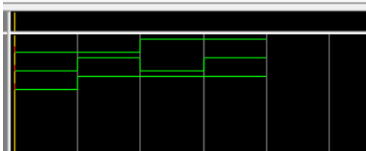
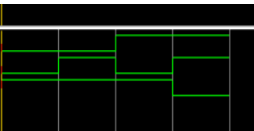
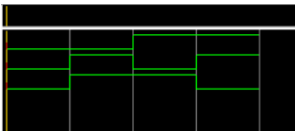
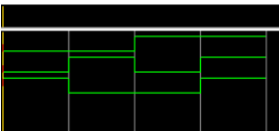
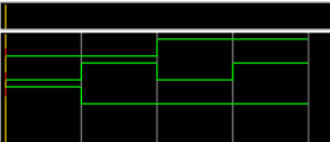
#50 x=1; y=0;

#50 x=1; y=1;

#50;

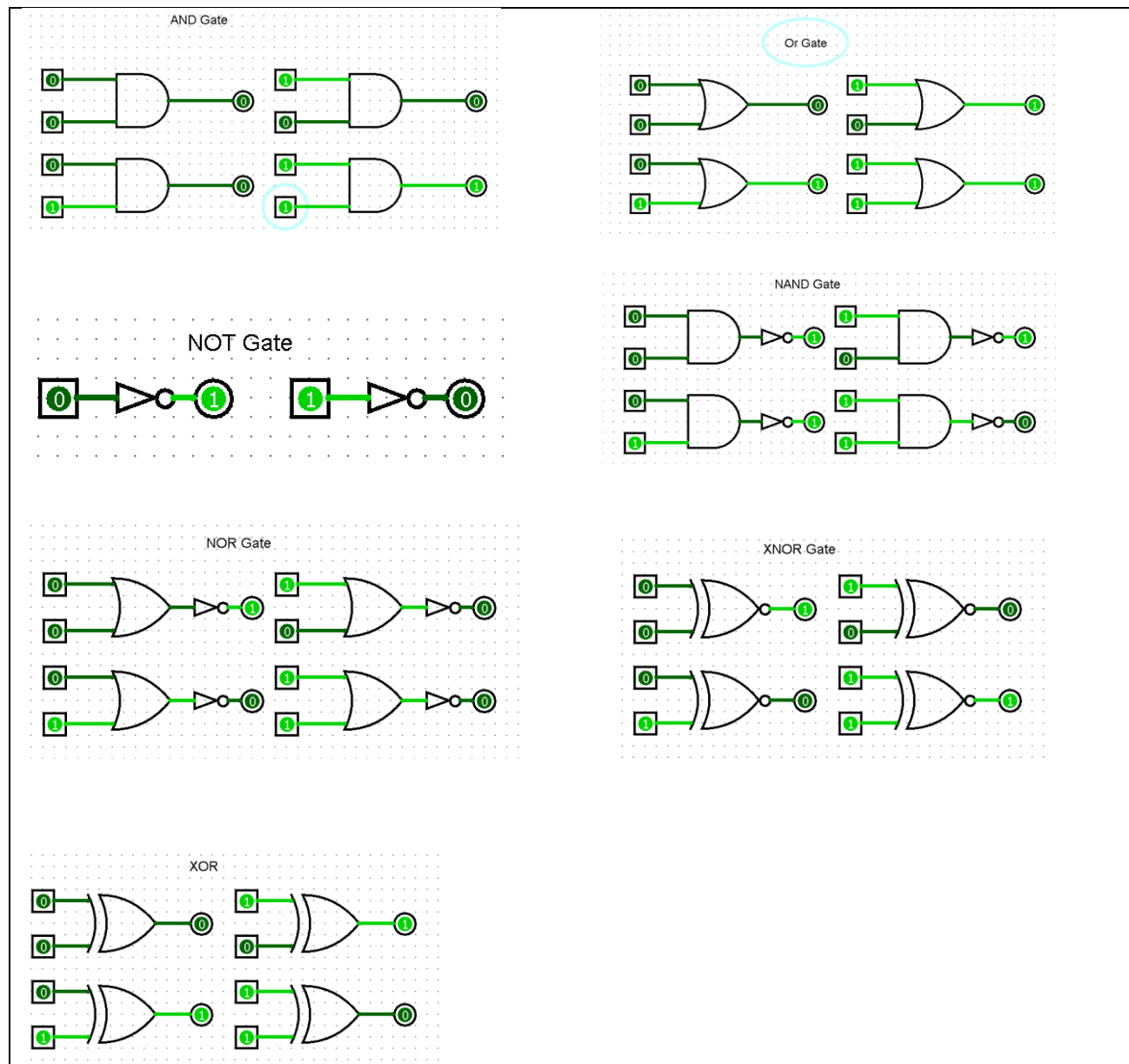
end

endmodule
```

Output:			
And Gate	Or Gate	Not Gate	
			
NAND Gate	Xor Gate	Xnor Gate	Nor Gate
			

Task 3:

Description:
In this task logic gates are tested using Logisim. Gates are to be connected with inputs and outputs to create a circuit. All the 4 input combinations were given in order to verify the output of the gates. These output matches the wave form of Modelsim.
Code:
No code for this Task.
Output:



Task 4:

Description:

The task requires implementing the given logic circuit in Verilog using gate-level modeling. The same circuit is also to be implemented with dataflow modeling using boolean expressions. All four combinations were provided for A and B and outputs of both implementations were simulated in wave form.

Code:

```
module complexGate(a,b,c);
```

```
input a;

input b;

output c;

wire t1;

wire t2;

wire t3;

wire t4;

not al1(t1,a);

and al2(t3,t1,b);

not al3(t2,b);

and al4(t4,a,t2);

or (c,t3,t4);

endmodule

module ComplexTestbench();

reg x;

reg y;

wire z;

complexGate uut (x,y,z);

initial

begin

x=0; y=0;

#50 x=0; y=1;

#50 x=1; y=0;

#50 x=1; y=1;

#50;
```

```
end

endmodule

//code with Data flow modeling:

module complexGate(a,b,c);

input a;

input b;

output c;

wire t1;

wire t2;

wire t3;

wire t4;

assign t1=~a;

assign t3=t1&b;

assign t2=~b;

assign t4=a&t2;

assign c=t3 | t4;

endmodule

module ComplexTestbench();

reg x;

reg y;

wire z;

complexGate uut (x,y,z);

initial

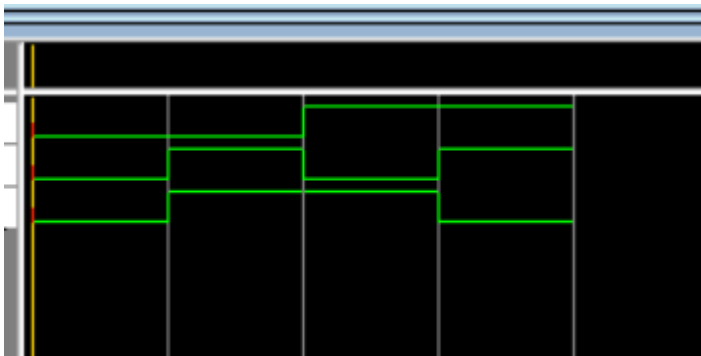
begin
```

```
x=0; y=0;  
  
#50 x=0; y=1;  
  
#50 x=1; y=0;  
  
#50 x=1; y=1;  
  
#50;  
  
end  
  
endmodule
```

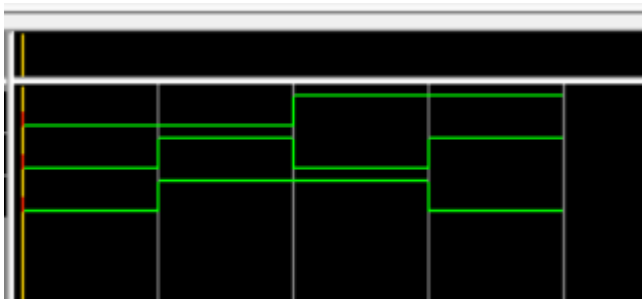
Output:

// Import output here 

Output with Gate level modeling



Output with Data flow modeling:



Task 5:

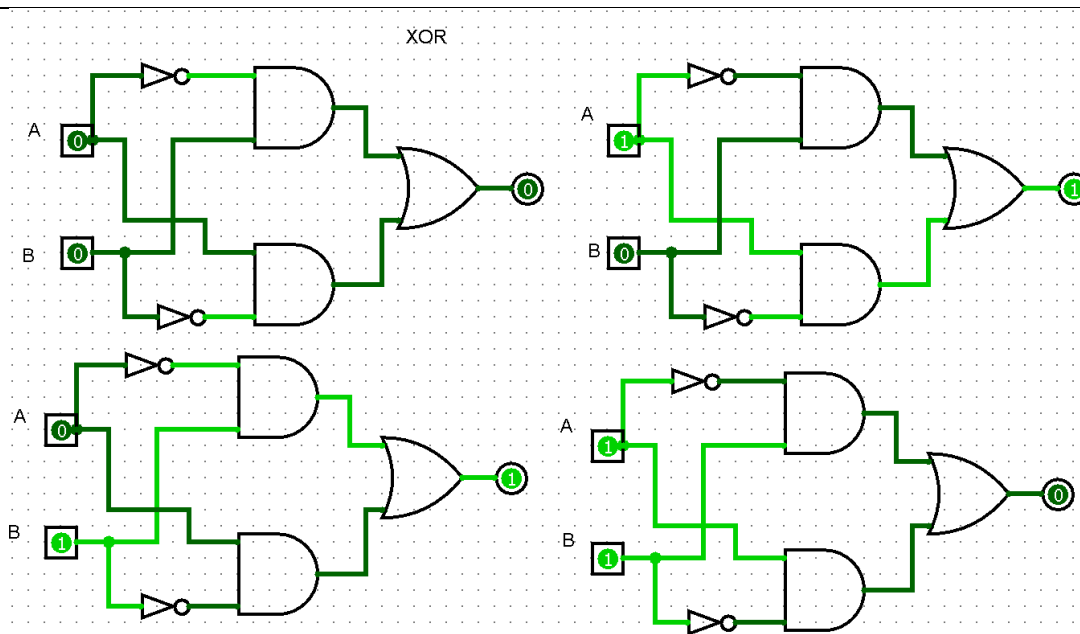
Description:

In this task I test the working of the given circuit using Logisim. The NOT, AND and OR gates are connected according to the given circuit diagram. Different input combinations are applied to check the output. The Logisim results were verified with the expected circuit behavior.

Code:

No code for this task

Output:



Task 6:

Description:

In this task I write Verilog code to find the 1's complement of a four-bit input. Gate level modeling is used by applying NOT gates to each input bit. The same operation is also implemented using dataflow modeling. A testbench module was used to verify the results for different four inputs.

Code:

```
//With Gate level modeling

module oneCompliment(a,b,c,d,w,x,y,z);

input a;

input b;

input c;

input d;

output w;

output x;

output y;

output z;

not g1(w,a);

not g2(x,b);

not g3(y,c);

not g4(z,d);

endmodule


module testbench();

reg a, b, c, d;

wire w, x, y, z;

oneCompliment uut(a,b,c,d,w,x,y,z);

initial
```

```
begin

a=0; b=0; c=0; d=0;

#50 a=0; b=0; c=0; d=1;

#50 a=0; b=0; c=1; d=0;

#50 a=0; b=0; c=1; d=1;

#50 a=0; b=1; c=0; d=0;

#50 a=0; b=1; c=0; d=1;

#50 a=0; b=1; c=1; d=0;

#50 a=0; b=1; c=1; d=1;

#50 a=1; b=0; c=0; d=0;

#50 a=1; b=0; c=0; d=1;

#50 a=1; b=0; c=1; d=0;

#50 a=1; b=0; c=1; d=1;

#50 a=1; b=1; c=0; d=0;

#50 a=1; b=1; c=0; d=1;

#50 a=1; b=1; c=1; d=0;

#50 a=1; b=1; c=1; d=1;

#50;

end

endmodule

//with Data flow model

module oneCompliment(a,b,c,d,w,x,y,z);

input a;

input b;

input c;

input d;

output w;
```

```
output x;

output y;

output z;

assign w=~a;

assign x=~b;

assign y=~c;

assign z=~d;

endmodule


module testbench();

reg a, b, c, d;

wire w, x, y, z;

oneCompliment uut(a,b,c,d,w,x,y,z);

initial

begin

a=0; b=0; c=0; d=0;

#50 a=0; b=0; c=0; d=1;

#50 a=0; b=0; c=1; d=0;

#50 a=0; b=0; c=1; d=1;

#50 a=0; b=1; c=0; d=0;

#50 a=0; b=1; c=0; d=1;

#50 a=0; b=1; c=1; d=0;

#50 a=0; b=1; c=1; d=1;

#50 a=1; b=0; c=0; d=0;

#50 a=1; b=0; c=0; d=1;

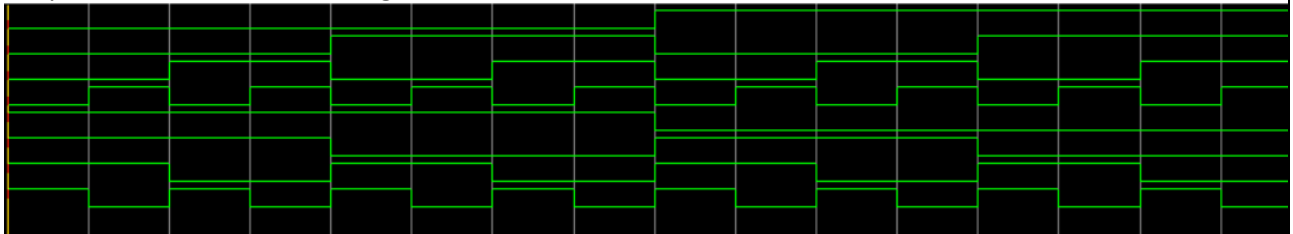
#50 a=1; b=0; c=1; d=0;

#50 a=1; b=0; c=1; d=1;
```

```
#50 a=1; b=1; c=0; d=0;  
  
#50 a=1; b=1; c=0; d=1;  
  
#50 a=1; b=1; c=1; d=0;  
  
#50 a=1; b=1; c=1; d=1;  
  
#50;  
  
end  
  
endmodule
```

Output:

Output with Gate level modeling



Output with Data Flow modeling

